UNIVERSIDAD DEL QUINDIO
PROGRAMA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN
BASES DE DATOS I

ENUNCIADO PROYECTO FINAL

El campeonato mundial de fútbol es un evento deportivo que se lleva a cabo cada cuatro años. Para este año, el mundial se jugará en México, USA y Canadá. Contará con 48 equipos provenientes de las diferentes confederaciones del mundo.

Se requiere crear una base de datos que permita almacenar la información de todos los equipos que van a jugar el mundial, sus directores técnicos y jugadores. Es necesario almacenar la información de las ciudades y de los estadios donde se jugarán los partidos de la fase de grupos; esto indica que, se deben tener almacenados los 12 grupos y sus partidos. La base de datos contará con una aplicación en un lenguaje de programación previamente establecido, así como el SGBD.

La aplicación que se creará será de escritorio y NO se van a utilizar frameworks para nada.

La aplicación deberá contar con un sistema básico de seguridad que consiste en un login de usuarios. Habrán tres tipos de usuarios: El Administrador (solo 1), los usuarios tradicionales y los usuarios esporádicos. Solo el Administrador puede crear nuevos usuarios. Los usuarios tradicionales pueden ejecutar los CRUD para la gestión de datos. Los usuarios esporádicos solo podrán ejecutar las consultas. También, se contará con una Bitácora que almacenará los datos del ingreso y la salida de los usuarios del sistema (fechas y horas de entrada y salida).

Consultas y reportes mínimos que debe tener la aplicación.

a) Consultas

- Determinar los datos del jugador más costoso por confederación
- Listar los partidos que se llevarán a cabo en un estadio cualquiera que el usuario elija.
- Determinar el equipo más costoso de los que van a jugar en cada país (México, USA, Canadá) en la fase de grupos.
- Determinar la cantidad de jugadores por equipo que tienen menos de 21 años.

b) Reportes (Archivo en .pdf o en una herramienta de reportes mostrado dentro de un formulario en la aplicación)

- Listar los usuarios que hayan ingresado y salido de la aplicación en una fecha y hora específica
- Listar los jugadores cuyo peso, estatura y equipo están dentro de lo solicitado por el usuario.
- Determinar el valor total de los jugadores por equipo y que pertenecen a una confederación específica.
- Listar los países que se jugarán en cada país anfitrión

1. Diseño de la Base de Datos (Relacional)

Para cumplir con los requisitos, propongo el siguiente esquema lógico. Es vital separar la logística (estadios/ciudades) de la deportiva (equipos/jugadores).

Tablas Principales:

- **Confederaciones:** (ID, Nombre).
- **Equipos:** (ID, Nombre, ID_Confederacion, ID_DT, Valor_Mercado).
- **Jugadores:** (ID, Nombre, Edad, Peso, Estatura, Valor, ID_Equipo).
- **Sedes:**
  - **Ciudades:** (ID, Nombre, Pais).
  - **Estadios:** (ID, Nombre, Capacidad, ID_Ciudad).
- **Torneo:**
  - **Grupos:** (ID, Letra/Nombre).
  - **Partidos:** (ID, ID_Equipo1, ID_Equipo2, ID_Estadio, ID_Grupo, Fecha_Hora).
- **Seguridad:**
  - **Usuarios:** (ID, Username, Password, Rol [Admin, Tradicional, Esporádico]).
  - **Bitacora:** (ID, ID_Usuario, Fecha_Ingreso, Fecha_Salida).

2. Lógica de Negocio y Seguridad

Al ser una aplicación de escritorio "vanilla" (sin frameworks), te recomiendo seguir el patrón **DAO (Data Access Object)** para interactuar con el SGBD (MySQL o PostgreSQL son ideales).

Control de Acceso (RBAC)

Implementarás una clase SessionManager que verifique el rol del usuario tras el login:

- **Admin:** Habilita el módulo de Gestión de Usuarios.
- **Tradicional:** Habilita botones de Guardar/Editar/Eliminar en los formularios.
- **Esporádico:** Solo habilita la pestaña de Consultas y Reportes (Botones CRUD en setEnabled(false)).

3. Resolución de Consultas Complejas (SQL)

Para las consultas que solicitaste, aquí tienes la lógica SQL base que deberías implementar en tus métodos:

Requerimiento	Lógica Sugerida
Jugador más costoso por Conf.	SELECT * FROM Jugadores WHERE valor IN (SELECT MAX(valor) FROM Jugadores GROUP BY id_confederacion)
Equipo más costoso	Cruzar Equipos con Partidos y Estadios filtrando por el país del

por País	estadio.
Jugadores < 21 años	SELECT equipo, COUNT(*) FROM Jugadores WHERE edad < 21 GROUP BY equipo

Fase 1: Diseño de la Base de Datos (Modelo Entidad-Relación)

Antes de escribir una sola línea de código, necesitamos definir cómo se almacenarán los datos. Usaremos SQL (por ejemplo, MySQL, PostgreSQL o SQLite) como SGBD.

Entidades Principales y sus Atributos:

1. **Usuario**
   - id_usuario (PK, INT, AUTO_INCREMENT)
   - nombre_usuario (VARCHAR, ÚNICO)
   - contrasena_hash (VARCHAR, para guardar la contraseña de forma segura)
   - tipo_usuario (ENUM: 'Administrador', 'Tradicional', 'Esporadico')
   - fecha_creacion (DATETIME)
2. **Bitacora**
   - id_registro (PK, INT, AUTO_INCREMENT)
   - id_usuario (FK, INT, referencia a Usuario)
   - fecha_hora_ingreso (DATETIME)
   - fecha_hora_salida (DATETIME, puede ser NULL hasta que el usuario salga)
3. **Confederacion**
   - id_confederacion (PK, INT, AUTO_INCREMENT)
   - nombre (VARCHAR, ej: 'UEFA', 'CONMEBOL', 'CONCACAF')
4. **Equipo**
   - id_equipo (PK, INT, AUTO_INCREMENT)
   - nombre (VARCHAR, ej: 'Brasil', 'Francia')
   - pais (VARCHAR)
   - id_confederacion (FK, INT, referencia a Confederacion)
   - valor_total_equipo (DECIMAL, opcional, se puede calcular con una consulta)
5. **DirectorTecnico**
   - id_dt (PK, INT, AUTO_INCREMENT)
   - nombre (VARCHAR)
   - nacionalidad (VARCHAR)
   - fecha_nacimiento (DATE)
   - id_equipo (FK, INT, ÚNICO, referencia a Equipo - Un DT dirige un solo equipo)
6. **Jugador**
   - id_jugador (PK, INT, AUTO_INCREMENT)
   - nombre (VARCHAR)
   - fecha_nacimiento (DATE)
   - posicion (VARCHAR)
   - peso (DECIMAL, en kg)
   - estatura (DECIMAL, en metros)
   - valor_mercado (DECIMAL, en euros o dólares)
   - id_equipo (FK, INT, referencia a Equipo)

7. **PaisAnfitrion**
    - id_pais_anfitrion (PK, INT, AUTO_INCREMENT)
    - nombre (VARCHAR, ej: 'México', 'Estados Unidos', 'Canadá')
8. **Ciudad**
    - id_ciudad (PK, INT, AUTO_INCREMENT)
    - nombre (VARCHAR)
    - id_pais_anfitrion (FK, INT, referencia a PaisAnfitrion)
9. **Estadio**
    - id_estadio (PK, INT, AUTO_INCREMENT)
    - nombre (VARCHAR)
    - capacidad (INT)
    - id_ciudad (FK, INT, referencia a Ciudad)
10. **Grupo**
    - id_grupo (PK, INT, AUTO_INCREMENT)
    - nombre_grupo (VARCHAR(1), ej: 'A', 'B', 'C', ... 'L') -> 12 grupos.
11. **Equipo_Grupo** (Tabla intermedia para la relación muchos a muchos entre Equipo y Grupo)
    - id_equipo_grupo (PK, INT, AUTO_INCREMENT)
    - id_equipo (FK, INT, referencia a Equipo)
    - id_grupo (FK, INT, referencia a Grupo)
12. **Partido**
    - id_partido (PK, INT, AUTO_INCREMENT)
    - id_equipo_local (FK, INT, referencia a Equipo)
    - id_equipo_visitante (FK, INT, referencia a Equipo)
    - fecha_hora (DATETIME)
    - id_estadio (FK, INT, referencia a Estadio)
    - id_grupo (FK, INT, referencia a Grupo) -> Para saber de qué grupo es el partido.

Diagrama simplificado de relaciones:

- Un Usuario tiene muchos registros en Bitacora.
- Una Confederacion tiene muchos Equipos.
- Un Equipo tiene un DirectorTecnico y muchos Jugadores.
- Un Equipo puede estar en un Grupo (a través de Equipo_Grupo).
- Un Grupo tiene muchos Equipos y muchos Partidos.
- Un PaisAnfitrion tiene muchas Ciudades.
- Una Ciudad tiene muchos Estadios.
- Un Estadio tiene muchos Partidos.
- Un Partido involucra a dos Equipos (local y visitante) y se juega en un Estadio.

1. **Cantidad de jugadores menores de 21 años por equipo:**
    - **SQL:** SELECT e.nombre, COUNT(j.id_jugador) as menores_21 FROM equipo e LEFT JOIN jugador j ON e.id_equipo = j.id_equipo WHERE DATE_PART('year', AGE(j.fecha_nacimiento)) < 21 GROUP BY e.nombre; (La función de fecha puede variar según el SGBD).
    - **Resultado:** Una tabla con dos columnas: "Equipo" y "Jugadores < 21".

Módulo 4: Reportes (Para Todos los Usuarios)

Estos reportes pueden mostrarse dentro de la aplicación (en una tabla) y además ofrecer un botón para "Exportar a PDF".

1. **Ingresos y Salidas de Usuarios en fecha/hora específica:**
    - **Vista:** Dos selectores de fecha y hora (o uno de rango de fechas) para que el usuario elija.
    - **Lógica:** Consultar la Bitacora unida con Usuario y filtrar por fecha_hora_ingreso y fecha_hora_salida según el rango.
    - **Reporte:** Mostrar en una tabla: Nombre de Usuario, Fecha/Hora Ingreso, Fecha/Hora Salida.
2. **Jugadores filtrados por peso, estatura y equipo:**
    - **Vista:** Campos de entrada para "Peso Mínimo", "Peso Máximo", "Estatura Mínima", "Estatura Máxima", y un JComboBox con la lista de equipos.
    - **Lógica:** Construir una consulta SQL dinámica que agregue las condiciones según lo que el usuario haya llenado. Si no selecciona equipo, se buscan en todos.
    - **Reporte:** Mostrar en una tabla los jugadores que cumplan los filtros.
3. **Valor total de jugadores por equipo y confederación específica:**
    - **Vista:** Un JComboBox con las confederaciones.
    - **SQL:** SELECT e.nombre as equipo, SUM(j.valor_mercado) as valor_total FROM equipo e JOIN jugador j ON e.id_equipo = j.id_equipo WHERE e.id_confederacion = ? GROUP BY e.nombre ORDER BY valor_total DESC;
    - **Reporte:** Mostrar en una tabla: Equipo, Valor Total.
4. **Países que se jugarán en cada país anfitrión:**
    - **Lógica:** Determinar, para México, USA y Canadá, qué selecciones nacionales juegan partidos en sus estadios durante la fase de grupos. Esto implica unir Partido con Estadio, Ciudad y PaisAnfitrion y luego listar los Equipos (locales y visitantes) distintos que aparecen.
    - **Vista:** Podría ser un reporte con una sección por cada país anfitrión (México, USA, Canadá) y la lista de países visitantes (o todos los equipos que juegan allí).

Fase 4: Consideraciones Clave y Buenas Prácticas

- **Sin Frameworks:** Todo el código de interfaz (Swing), acceso a datos (JDBC) y lógica debe ser escrito por ti. Evita usar Hibernate, Spring, etc.
- **Seguridad:**
    - **Contraseñas:** Nunca guardes contraseñas en texto plano. Usa hashing con un salt (ej: bcrypt).
    - **Inyección SQL:** Usa siempre PreparedStatement en JDBC para evitar inyecciones SQL.
    - **Control de Acceso:** Antes de cada acción en la capa de servicios/vista, verifica el tipo_usuario del usuario en sesión.
- **Manejo de la Sesión:** Crea una clase SesionUsuario (Singleton) que mantenga el objeto Usuario actual y su id_registro_bitacora activo. Así puedes acceder a esta información desde cualquier parte de la aplicación.
- **Bitácora:** El registro de entrada se hace al validar el login. El de salida se debe actualizar en un bloque finally o al cerrar la ventana principal para asegurar que siempre se ejecute.
- **Generación de PDF:** Investiga la librería iText. Deberás tomar los datos de los resultados de las consultas y escribir el PDF con tablas y estilos básicos.

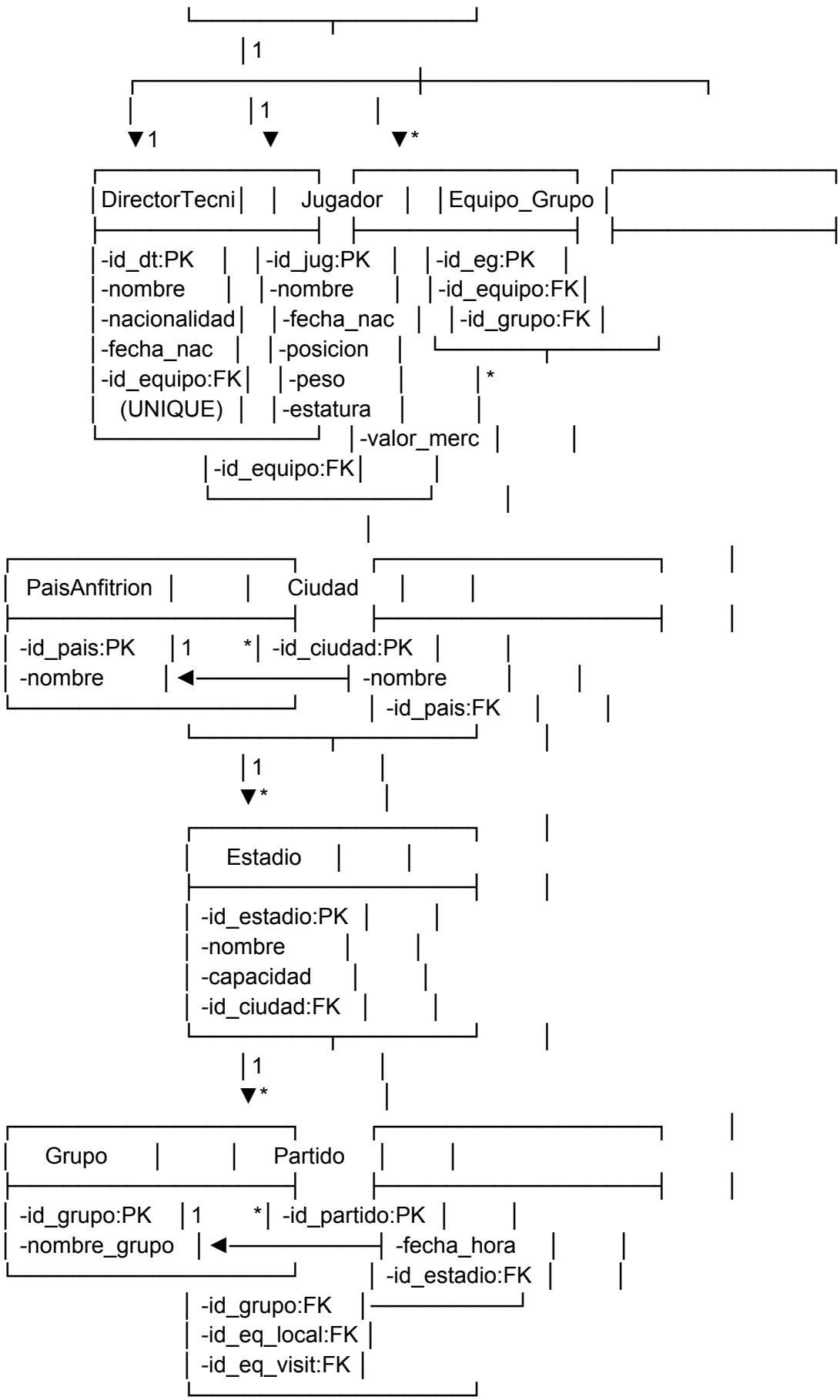
- **Interfaz de Usuario:** Diseña menús (JMenuBar) que se habiliten o deshabiliten según el tipo de usuario. Por ejemplo, el menú "Administración" > "Usuarios" solo visible para el Admin.

Ejemplo de Flujo de Trabajo:

1. El usuario inicia la app y ve la ventana de Login.
2. Ingresa sus credenciales. El sistema verifica en UsuarioDAO.
3. Si es correcto, se inicia la SesionUsuario, se crea un registro en Bitacora y se abre la ventana principal.
4. La ventana principal tiene un JMenuBar. Dependiendo del tipo de usuario en SesionUsuario, se habilitan diferentes opciones de menú.
5. Si es un usuario Tradicional, puede hacer clic en "Gestión" -> "Equipos". Se abre una ventana con una tabla de equipos y botones para agregar, editar, eliminar. Al hacer clic en "Agregar", se abre un formulario y al guardar, se llama a EquipoDAO.insertar(equipo).
6. Si es un usuario Esporadico, hace clic en "Consultas" -> "Jugador más costoso por confederación". Se abre una ventana con un JComboBox. Al seleccionar una y hacer clic en "Buscar", se ejecuta la lógica en JugadorDAO y se muestran los resultados.
7. Al salir de la aplicación, se captura el evento de cierre y se actualiza el registro de Bitacora con la hora de salida.

```
┌──────────────────────┐        ┌──────────────────────┐
│   Usuario            │        │   Bitacora           │
├──────────────────────┤        ├──────────────────────┤
│ -id_usuario:PK       │1      *│ -id_registro:PK      │
│ -nombre_usuario      │◀───────┤ -fecha_entrada       │
│ -contrasena_hash     │        │ -fecha_salida        │
│ -tipo_usuario        │        │ -id_usuario:FK       │
│ -fecha_creacion      │        └──────────────────────┘
└──────────────────────┘

┌──────────────────────┐        ┌──────────────────────┐
│  Confederacion       │        │   Equipo             │
├──────────────────────┤        ├──────────────────────┤
│ -id_confed:PK        │1      *│ -id_equipo:PK        │
│ -nombre              │◀───────┤ -nombre              │
└──────────────────────┘        │ -pais                │
                                │ -valor_total         │
                                │ -id_confed:FK        │
```


1
1
1
*
DirectorTecni
-id_dt:PK
-nombre
-nacionalidad
-fecha_nac
-id_equipo:FK
(UNIQUE)
Jugador
-id_jug:PK
-nombre
-fecha_nac
-posicion
-peso
-estatura
-valor_merc
-id_equipo:FK
Equipo_Grupo
-id_eg:PK
-id_equipo:FK
-id_grupo:FK
*
PaisAnfitrion
-id_pais:PK
-nombre
1
*
Ciudad
-id_ciudad:PK
-nombre
-id_pais:FK
1
*
Estadio
-id_estadio:PK
-nombre
-capacidad
-id_ciudad:FK
1
*
Grupo
-id_grupo:PK
-nombre_grupo
1
*
Partido
-id_partido:PK
-fecha_hora
-id_estadio:FK
-id_grupo:FK
-id_eq_local:FK
-id_eq_visit:FK

RESTRICCIONES DE INTEGRIDAD IMPORTANTES

1. **Integridad Referencial**: Todas las FK deben apuntar a PKs existentes
2. **UNIQUE en DT.id_equipo**: Garantiza relación 1:1
3. **CHECK en Jugador**: Edad mínima, peso y estatura en rangos lógicos
4. **CHECK en Partido**: id_equipo_local ≠ id_equipo_visitante
5. **Dependencia de existencia**: Bitácora depende de Usuario (ON DELETE CASCADE?)
6. **Cardinalidad de grupos**: Exactamente 12 grupos, 4 equipos por grupo

```
-- ====================================================

-- CREACIÓN DE LA BASE DE DATOS

-- ====================================================

CREATE DATABASE IF NOT EXISTS Mundial2026;

USE Mundial2026;


-- ====================================================

-- TABLAS PRINCIPALES (SIN DEPENDENCIAS EXTERNAS)

-- ====================================================


-- Tabla de Usuarios

CREATE TABLE Usuario (

    id_usuario INT PRIMARY KEY AUTO_INCREMENT,

    nombre_usuario VARCHAR(50) UNIQUE NOT NULL,

    contrasena_hash VARCHAR(255) NOT NULL,

    tipo_usuario ENUM('Administrador', 'Tradicional', 'Esporadico') NOT NULL,

    fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP

);


-- Tabla de Confederaciones

CREATE TABLE Confederacion (
```

```
    id_confederacion INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100) UNIQUE NOT NULL,
    siglas VARCHAR(10) NOT NULL
);

-- Tabla de Países Anfitriones
CREATE TABLE PaisAnfitrion (
    id_pais_anfitrion INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100) UNIQUE NOT NULL
);

-- Tabla de Grupos
CREATE TABLE Grupo (
    id_grupo INT PRIMARY KEY AUTO_INCREMENT,
    nombre_grupo VARCHAR(1) NOT NULL,
    CONSTRAINT chk_grupo CHECK (nombre_grupo IN ('A','B','C','D','E','F','G','H','I','J','K','L'))
);

-- ====================================================
-- TABLAS CON DEPENDENCIAS
-- ====================================================

-- Tabla de Equipos (depende de Confederacion)
CREATE TABLE Equipo (
    id_equipo INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100) UNIQUE NOT NULL,
    pais VARCHAR(100) NOT NULL,
```

```
    valor_total_equipo DECIMAL(15,2) DEFAULT 0,
    id_confederacion INT NOT NULL,
    FOREIGN KEY (id_confederacion) REFERENCES Confederacion(id_confederacion)
);

-- Tabla de Directores Técnicos (depende de Equipo)
CREATE TABLE DirectorTecnico (
    id_dt INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(200) NOT NULL,
    nacionalidad VARCHAR(100) NOT NULL,
    fecha_nacimiento DATE NOT NULL,
    id_equipo INT UNIQUE NOT NULL,
    FOREIGN KEY (id_equipo) REFERENCES Equipo(id_equipo)
);

-- Tabla de Jugadores (depende de Equipo)
CREATE TABLE Jugador (
    id_jugador INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(200) NOT NULL,
    fecha_nacimiento DATE NOT NULL,
    posicion VARCHAR(50) NOT NULL,
    peso DECIMAL(5,2) NOT NULL,
    estatura DECIMAL(3,2) NOT NULL,
    valor_mercado DECIMAL(15,2) NOT NULL,
    id_equipo INT NOT NULL,
    FOREIGN KEY (id_equipo) REFERENCES Equipo(id_equipo),
    CONSTRAINT chk_peso CHECK (peso > 0 AND peso < 200),
```

```
    CONSTRAINT chk_estatura CHECK (estatura > 0 AND estatura < 2.50),
    CONSTRAINT chk_valor CHECK (valor_mercado >= 0)
);

-- Tabla de Ciudades (depende de PaisAnfitrion)
CREATE TABLE Ciudad (
    id_ciudad INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    id_pais_anfitrion INT NOT NULL,
    FOREIGN KEY (id_pais_anfitrion) REFERENCES PaisAnfitrion(id_pais_anfitrion),
    UNIQUE KEY unique_ciudad_pais (nombre, id_pais_anfitrion)
);

-- Tabla de Estadios (depende de Ciudad)
CREATE TABLE Estadio (
    id_estadio INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100) UNIQUE NOT NULL,
    capacidad INT NOT NULL,
    id_ciudad INT NOT NULL,
    FOREIGN KEY (id_ciudad) REFERENCES Ciudad(id_ciudad),
    CONSTRAINT chk_capacidad CHECK (capacidad > 0)
);

-- ====================================================
-- TABLAS INTERMEDIAS Y DEPENDENCIAS CRUZADAS
-- ====================================================
```

```
-- Tabla intermedia Equipo-Grupo (relación N:M)
CREATE TABLE Equipo_Grupo (
    id_equipo_grupo INT PRIMARY KEY AUTO_INCREMENT,
    id_equipo INT NOT NULL,
    id_grupo INT NOT NULL,
    FOREIGN KEY (id_equipo) REFERENCES Equipo(id_equipo),
    FOREIGN KEY (id_grupo) REFERENCES Grupo(id_grupo),
    UNIQUE KEY unique_equipo_grupo (id_equipo, id_grupo)
);

-- Tabla de Partidos (depende de Equipo, Estadio y Grupo)
CREATE TABLE Partido (
    id_partido INT PRIMARY KEY AUTO_INCREMENT,
    fecha_hora DATETIME NOT NULL,
    id_estadio INT NOT NULL,
    id_grupo INT NOT NULL,
    id_equipo_local INT NOT NULL,
    id_equipo_visitante INT NOT NULL,
    goles_local INT DEFAULT 0,
    goles_visitante INT DEFAULT 0,
    FOREIGN KEY (id_estadio) REFERENCES Estadio(id_estadio),
    FOREIGN KEY (id_grupo) REFERENCES Grupo(id_grupo),
    FOREIGN KEY (id_equipo_local) REFERENCES Equipo(id_equipo),
    FOREIGN KEY (id_equipo_visitante) REFERENCES Equipo(id_equipo),
    CONSTRAINT chk_equipos_diferentes CHECK (id_equipo_local != id_equipo_visitante),
    CONSTRAINT chk_goles CHECK (goles_local >= 0 AND goles_visitante >= 0)
);
```

```
-- Tabla de Bitácora (depende de Usuario)
CREATE TABLE Bitacora (
    id_registro INT PRIMARY KEY AUTO_INCREMENT,
    id_usuario INT NOT NULL,
    fecha_hora_ingreso DATETIME DEFAULT CURRENT_TIMESTAMP,
    fecha_hora_salida DATETIME NULL,
    FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario)
);

-- ====================================================
-- ÍNDICES PARA OPTIMIZAR CONSULTAS
-- ====================================================

CREATE INDEX idx_jugador_equipo ON Jugador(id_equipo);
CREATE INDEX idx_jugador_valor ON Jugador(valor_mercado);
CREATE INDEX idx_jugador_edad ON Jugador(fecha_nacimiento);
CREATE INDEX idx_partido_fecha ON Partido(fecha_hora);
CREATE INDEX idx_partido_estadio ON Partido(id_estadio);
CREATE INDEX idx_bitacora_fechas ON Bitacora(fecha_hora_ingreso, fecha_hora_salida);
CREATE INDEX idx_bitacora_usuario ON Bitacora(id_usuario);

-- ====================================================
-- INSERTS DE EJEMPLO PARA PRUEBAS
-- ====================================================

-- Insertar Usuario Administrador (contraseña: admin123)
```

```
INSERT INTO Usuario (nombre_usuario, contrasena_hash, tipo_usuario) VALUES

('admin', '8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918', 'Administrador');


-- Insertar Confederaciones

INSERT INTO Confederacion (nombre, siglas) VALUES

('Unión de Federaciones Europeas de Fútbol', 'UEFA'),

('Confederación Sudamericana de Fútbol', 'CONMEBOL'),

('Confederación de Fútbol de Norte, Centroamérica y el Caribe', 'CONCACAF'),

('Confederación Africana de Fútbol', 'CAF'),

('Confederación Asiática de Fútbol', 'AFC'),

('Confederación de Fútbol de Oceanía', 'OFC');


-- Insertar Países Anfitriones

INSERT INTO PaisAnfitrion (nombre) VALUES

('México'),

('Estados Unidos'),

('Canadá');


-- Insertar Grupos (12 grupos)

INSERT INTO Grupo (nombre_grupo) VALUES

('A'), ('B'), ('C'), ('D'), ('E'), ('F'),

('G'), ('H'), ('I'), ('J'), ('K'), ('L');


-- Insertar Ciudades y Estadios de ejemplo

INSERT INTO Ciudad (nombre, id_pais_anfitrion) VALUES

('Ciudad de México', 1),

('Guadalajara', 1),
```

```
('Monterrey', 1),
('Los Ángeles', 2),
('Nueva York', 2),
('Miami', 2),
('Toronto', 3),
('Vancouver', 3),
('Montreal', 3);

INSERT INTO Estadio (nombre, capacidad, id_ciudad) VALUES
('Estadio Azteca', 87523, 1),
('Estadio Akron', 49850, 2),
('Estadio BBVA', 53500, 3),
('Rose Bowl', 92542, 4),
('MetLife Stadium', 82500, 5),
('Hard Rock Stadium', 65326, 6),
('BMO Field', 30991, 7),
('BC Place', 54500, 8),
('Stade Olympique', 56040, 9);

-- Insertar algunos Equipos de ejemplo
INSERT INTO Equipo (nombre, pais, valor_total_equipo, id_confederacion) VALUES
('Brasil', 'Brasil', 850000000, 2),
('Argentina', 'Argentina', 780000000, 2),
('Francia', 'Francia', 920000000, 1),
('España', 'España', 750000000, 1),
('México', 'México', 280000000, 3),
('Estados Unidos', 'Estados Unidos', 320000000, 3),
```

('Canadá', 'Canadá', 180000000, 3),

('Japón', 'Japón', 150000000, 5);

-- Insertar Directores Técnicos

INSERT INTO DirectorTecnico (nombre, nacionalidad, fecha_nacimiento, id_equipo) VALUES

('Tite', 'Brasileña', '1961-05-25', 1),

('Lionel Scaloni', 'Argentina', '1978-05-16', 2),

('Didier Deschamps', 'Francesa', '1968-10-15', 3),

('Luis de la Fuente', 'Española', '1961-06-21', 4),

('Jaime Lozano', 'Mexicana', '1978-09-29', 5),

('Gregg Berhalter', 'Estadounidense', '1973-08-01', 6),

('John Herdman', 'Inglesa', '1975-07-19', 7),

('Hajime Moriyasu', 'Japonesa', '1968-08-23', 8);

-- Insertar Jugadores de ejemplo

INSERT INTO Jugador (nombre, fecha_nacimiento, posicion, peso, estatura, valor_mercado, id_equipo) VALUES

('Neymar Jr', '1992-02-05', 'Delantero', 68.5, 1.75, 90000000, 1),

('Vinicius Jr', '2000-07-12', 'Delantero', 73.0, 1.76, 150000000, 1),

('Lionel Messi', '1987-06-24', 'Delantero', 72.0, 1.70, 50000000, 2),

('Enzo Fernández', '2001-01-17', 'Centrocampista', 78.0, 1.78, 75000000, 2),

('Kylian Mbappé', '1998-12-20', 'Delantero', 73.0, 1.78, 180000000, 3),

('Eduardo Camavinga', '2002-11-10', 'Centrocampista', 68.0, 1.82, 85000000, 3),

('Pedri', '2002-11-25', 'Centrocampista', 60.0, 1.74, 90000000, 4),

('Gavi', '2004-08-05', 'Centrocampista', 70.0, 1.73, 80000000, 4),

('Santiago Giménez', '2001-04-18', 'Delantero', 76.0, 1.82, 40000000, 5),

('Edson Álvarez', '1997-10-24', 'Defensa', 75.0, 1.87, 35000000, 5),

('Christian Pulisic', '1998-09-18', 'Delantero', 69.0, 1.78, 45000000, 6),

```
('Weston McKennie', '1998-08-28', 'Centrocampista', 81.0, 1.85, 25000000, 6),
('Alphonso Davies', '2000-11-02', 'Defensa', 75.0, 1.83, 70000000, 7),
('Jonathan David', '2000-01-14', 'Delantero', 70.0, 1.75, 45000000, 7),
('Takefusa Kubo', '2001-06-04', 'Centrocampista', 67.0, 1.73, 30000000, 8),
('Ritsu Doan', '1998-06-16', 'Centrocampista', 70.0, 1.72, 20000000, 8);

-- Asignar equipos a grupos
INSERT INTO Equipo_Grupo (id_equipo, id_grupo) VALUES
(1, 1), (2, 1), (3, 2), (4, 2),
(5, 3), (6, 3), (7, 4), (8, 4);

-- Insertar algunos partidos de ejemplo
INSERT INTO Partido (fecha_hora, id_estadio, id_grupo, id_equipo_local, id_equipo_visitante) VALUES
('2026-06-14 15:00:00', 1, 1, 1, 2),
('2026-06-15 18:00:00', 4, 2, 3, 4),
('2026-06-16 20:00:00', 7, 3, 5, 6),
('2026-06-17 14:00:00', 2, 4, 7, 8);

-- ====================================================
-- VISTAS ÚTILES PARA CONSULTAS
-- ====================================================

-- Vista para obtener información completa de jugadores
CREATE VIEW vw_jugadores_completo AS
SELECT
    j.*,
    e.nombre AS equipo_nombre,
```

```
    e.pais AS equipo_pais,
    c.nombre AS confederacion,
    c.siglas AS confederacion_siglas
FROM Jugador j
INNER JOIN Equipo e ON j.id_equipo = e.id_equipo
INNER JOIN Confederacion c ON e.id_confederacion = c.id_confederacion;

-- Vista para información de partidos
CREATE VIEW vw_partidos_completo AS
SELECT
    p.id_partido,
    p.fecha_hora,
    el.nombre AS equipo_local,
    ev.nombre AS equipo_visitante,
    est.nombre AS estadio,
    ci.nombre AS ciudad,
    pa.nombre AS pais_anfitrion,
    g.nombre_grupo AS grupo,
    p.goles_local,
    p.goles_visitante
FROM Partido p
INNER JOIN Equipo el ON p.id_equipo_local = el.id_equipo
INNER JOIN Equipo ev ON p.id_equipo_visitante = ev.id_equipo
INNER JOIN Estadio est ON p.id_estadio = est.id_estadio
INNER JOIN Ciudad ci ON est.id_ciudad = ci.id_ciudad
INNER JOIN PaisAnfitrion pa ON ci.id_pais_anfitrion = pa.id_pais_anfitrion
INNER JOIN Grupo g ON p.id_grupo = g.id_grupo;
```

```
-- ==========================================================
-- PROCEDIMIENTOS ALMACENADOS PARA CONSULTAS ESPECÍFICAS
-- ==========================================================

-- 1. Jugador más costoso por confederación
DELIMITER //
CREATE PROCEDURE sp_jugador_mas_costoso_por_confederacion()
BEGIN
    SELECT
        c.nombre AS confederacion,
        j.nombre AS jugador,
        e.nombre AS equipo,
        j.valor_mercado,
        j.posicion,
        TIMESTAMPDIFF(YEAR, j.fecha_nacimiento, CURDATE()) AS edad
    FROM Jugador j
    INNER JOIN Equipo e ON j.id_equipo = e.id_equipo
    INNER JOIN Confederacion c ON e.id_confederacion = c.id_confederacion
    WHERE (c.id_confederacion, j.valor_mercado) IN (
        SELECT
            e.id_confederacion,
            MAX(j.valor_mercado)
        FROM Jugador j
        INNER JOIN Equipo e ON j.id_equipo = e.id_equipo
        GROUP BY e.id_confederacion
    )
```

```
    ORDER BY j.valor_mercado DESC;
END//
DELIMITER ;

-- 2. Partidos por estadio
DELIMITER //
CREATE PROCEDURE sp_partidos_por_estadio(IN estadio_nombre VARCHAR(100))
BEGIN
    SELECT
        p.fecha_hora,
        el.nombre AS equipo_local,
        ev.nombre AS equipo_visitante,
        g.nombre_grupo AS grupo
    FROM Partido p
    INNER JOIN Equipo el ON p.id_equipo_local = el.id_equipo
    INNER JOIN Equipo ev ON p.id_equipo_visitante = ev.id_equipo
    INNER JOIN Estadio e ON p.id_estadio = e.id_estadio
    INNER JOIN Grupo g ON p.id_grupo = g.id_grupo
    WHERE e.nombre = estadio_nombre
    ORDER BY p.fecha_hora;
END//
DELIMITER ;

-- 3. Equipo más costoso por país anfitrión
DELIMITER //
CREATE PROCEDURE sp_equipo_mas_costoso_por_pais_anfitrion()
BEGIN
```

```
SELECT
    pa.nombre AS pais_anfitrion,
    e.nombre AS equipo,
    SUM(j.valor_mercado) AS valor_total_equipo,
    COUNT(j.id_jugador) AS num_jugadores
FROM PaisAnfitrion pa
INNER JOIN Ciudad ci ON pa.id_pais_anfitrion = ci.id_pais_anfitrion
INNER JOIN Estadio es ON ci.id_ciudad = es.id_ciudad
INNER JOIN Partido p ON es.id_estadio = p.id_estadio
INNER JOIN Equipo e ON (p.id_equipo_local = e.id_equipo OR p.id_equipo_visitante = e.id_equipo)
INNER JOIN Jugador j ON e.id_equipo = j.id_equipo
GROUP BY pa.id_pais_anfitrion, e.id_equipo
HAVING SUM(j.valor_mercado) = (
    SELECT MAX(total_equipo)
    FROM (
        SELECT
            pa2.id_pais_anfitrion,
            SUM(j2.valor_mercado) AS total_equipo
        FROM PaisAnfitrion pa2
        INNER JOIN Ciudad ci2 ON pa2.id_pais_anfitrion = ci2.id_pais_anfitrion
        INNER JOIN Estadio es2 ON ci2.id_ciudad = es2.id_ciudad
        INNER JOIN Partido p2 ON es2.id_estadio = p2.id_estadio
            INNER JOIN Equipo e2 ON (p2.id_equipo_local = e2.id_equipo OR p2.id_equipo_visitante =
e2.id_equipo)
        INNER JOIN Jugador j2 ON e2.id_equipo = j2.id_equipo
        GROUP BY pa2.id_pais_anfitrion, e2.id_equipo
    ) AS subquery
    WHERE subquery.id_pais_anfitrion = pa.id_pais_anfitrion
```

```
    );
END//
DELIMITER ;

-- 4. Jugadores menores de 21 años por equipo
DELIMITER //
CREATE PROCEDURE sp_jugadores_menores_21_por_equipo()
BEGIN
    SELECT
        e.nombre AS equipo,
        COUNT(j.id_jugador) AS jugadores_menores_21,
        GROUP_CONCAT(j.nombre SEPARATOR ', ') AS nombres_jugadores
    FROM Equipo e
    LEFT JOIN Jugador j ON e.id_equipo = j.id_equipo
        AND TIMESTAMPDIFF(YEAR, j.fecha_nacimiento, CURDATE()) < 21
    GROUP BY e.id_equipo, e.nombre
    ORDER BY jugadores_menores_21 DESC;
END//
DELIMITER ;

-- ======================================================
-- TRIGGERS PARA MANTENER INTEGRIDAD
-- ======================================================

-- Trigger para actualizar valor_total_equipo cuando se inserta/modifica un jugador
DELIMITER //
CREATE TRIGGER trg_actualizar_valor_equipo_insert
```

```
AFTER INSERT ON Jugador
FOR EACH ROW
BEGIN
    UPDATE Equipo
    SET valor_total_equipo = (
        SELECT SUM(valor_mercado)
        FROM Jugador
        WHERE id_equipo = NEW.id_equipo
    )
    WHERE id_equipo = NEW.id_equipo;
END//

CREATE TRIGGER trg_actualizar_valor_equipo_update
AFTER UPDATE ON Jugador
FOR EACH ROW
BEGIN
    IF NEW.id_equipo != OLD.id_equipo OR NEW.valor_mercado != OLD.valor_mercado THEN
        -- Actualizar equipo nuevo
        UPDATE Equipo
        SET valor_total_equipo = (
            SELECT SUM(valor_mercado)
            FROM Jugador
            WHERE id_equipo = NEW.id_equipo
        )
        WHERE id_equipo = NEW.id_equipo;

        -- Actualizar equipo antiguo
```

```
        UPDATE Equipo
        SET valor_total_equipo = (
            SELECT SUM(valor_mercado)
            FROM Jugador
            WHERE id_equipo = OLD.id_equipo
        )
        WHERE id_equipo = OLD.id_equipo;
    END IF;
END//

CREATE TRIGGER trg_actualizar_valor_equipo_delete
AFTER DELETE ON Jugador
FOR EACH ROW
BEGIN
    UPDATE Equipo
    SET valor_total_equipo = (
        SELECT IFNULL(SUM(valor_mercado), 0)
        FROM Jugador
        WHERE id_equipo = OLD.id_equipo
    )
    WHERE id_equipo = OLD.id_equipo;
END//
DELIMITER ;

-- ====================================================
-- FUNCIONES ÚTILES
-- ====================================================
```

```
DELIMITER //

CREATE FUNCTION f_calcular_edad_jugador(id_jug INT)

RETURNS INT

DETERMINISTIC

READS SQL DATA

BEGIN

   DECLARE edad INT;

   SELECT TIMESTAMPDIFF(YEAR, fecha_nacimiento, CURDATE()) INTO edad

   FROM Jugador WHERE id_jugador = id_jug;

   RETURN edad;

END//


DELIMITER ;


-- ====================================================

-- EJEMPLOS DE CONSULTAS PARA LOS REPORTES

-- ====================================================


-- 1. Usuarios que ingresaron/salieron en fecha específica

SELECT

   u.nombre_usuario,

   u.tipo_usuario,

   b.fecha_hora_ingreso,

   b.fecha_hora_salida,

   TIMEDIFF(b.fecha_hora_salida, b.fecha_hora_ingreso) AS tiempo_sesion

FROM Bitacora b
```

```
INNER JOIN Usuario u ON b.id_usuario = u.id_usuario
WHERE DATE(b.fecha_hora_ingreso) = '2026-06-14'
ORDER BY b.fecha_hora_ingreso;

-- 2. Jugadores filtrados por peso, estatura y equipo
SELECT
    j.nombre,
    e.nombre AS equipo,
    j.posicion,
    j.peso,
    j.estatura,
    j.valor_mercado,
    f_calcular_edad_jugador(j.id_jugador) AS edad
FROM Jugador j
INNER JOIN Equipo e ON j.id_equipo = e.id_equipo
WHERE j.peso BETWEEN 60 AND 80
    AND j.estatura BETWEEN 1.70 AND 1.85
    AND e.nombre = 'Brasil'
ORDER BY j.valor_mercado DESC;

-- 3. Valor total de jugadores por equipo y confederación específica
SELECT
    e.nombre AS equipo,
    COUNT(j.id_jugador) AS num_jugadores,
    SUM(j.valor_mercado) AS valor_total_plantilla,
    AVG(j.valor_mercado) AS valor_promedio_jugador
FROM Equipo e
```

```
INNER JOIN Confederacion c ON e.id_confederacion = c.id_confederacion

LEFT JOIN Jugador j ON e.id_equipo = j.id_equipo

WHERE c.nombre = 'CONMEBOL'

GROUP BY e.id_equipo, e.nombre

ORDER BY valor_total_plantilla DESC;


-- 4. Países que jugarán en cada país anfitrión

SELECT

    pa.nombre AS pais_anfitrion,

    e.nombre AS equipo_visitante,

    COUNT(DISTINCT p.id_partido) AS partidos_jugara

FROM PaisAnfitrion pa

INNER JOIN Ciudad ci ON pa.id_pais_anfitrion = ci.id_pais_anfitrion

INNER JOIN Estadio es ON ci.id_ciudad = es.id_ciudad

INNER JOIN Partido p ON es.id_estadio = p.id_estadio

INNER JOIN Equipo e ON (p.id_equipo_visitante = e.id_equipo)

WHERE e.pais != pa.nombre  -- Excluir equipos del país anfitrión

GROUP BY pa.id_pais_anfitrion, e.id_equipo

ORDER BY pa.nombre, partidos_jugara DESC;
```

CARACTERÍSTICAS IMPORTANTES DE ESTE SCRIPT:

1. **Integridad Referencial**: Todas las tablas tienen sus FOREIGN KEY bien definidas
2. **Restricciones CHECK**: Validan datos como peso, estatura, goles no negativos, etc.
3. **Índices Estratégicos**: Optimizan las consultas más frecuentes
4. **Vistas Útiles**: Simplifican consultas complejas
5. **Procedimientos Almacenados**: Implementan las consultas requeridas
6. **Triggers**: Mantienen automáticamente el valor total de los equipos
7. **Funciones**: Reutilizan lógica común como el cálculo de edad
8. **Datos de Ejemplo**: Permiten probar la base de datos inmediatamente